Programação Concorrente e Distribuída

Threads Virtuais

Glêdson Elias
gledson@ci.ufpb.br

1

Threads Java

- Características
  - Representam múltiplas linhas de execução que coexistem em um único processo
  - Podem ser executadas de forma concorrente
  - Compartilham recursos de um processo
    - Espaço de endereçamento do processo
  - Cada thread possui seu contador de programa, pilha e variáveis locais
  - Comunicação pode ocorrer via memória compartilhada



2

Threads Java

- Evolução
  - Projeto Loom/Oracle foi responsável pela proposta da nova arquitetura de threads Java
    - Threads de Plataforma (Platform Threads)
    - Threads Virtuais (Virtual Threads)
  - Incorporada e melhorada no Java 19/20 como recurso de pré-visualização / pré-estreia (preview feature)
  - Lançada oficialmente como recurso padrão no Java 21
  - Evoluiu em desempenho no Java 24/25

Threads de Plataforma

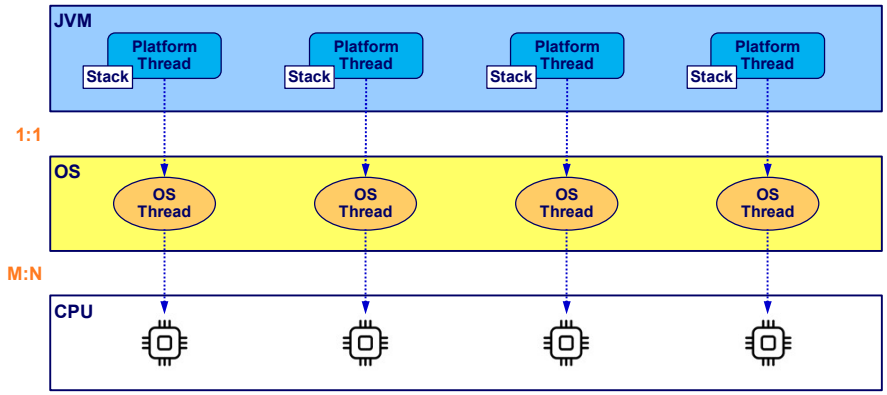
- Características
  - Representam os threads convencionais ou clássicos da arquitetura Java
  - Adequados à execução de todo tipo de tarefa
  - Podem tornar-se um recurso limitado
    - Alto consumo de memória e processamento
    - Contenção do escalonador
    - Limitados a alguns milhares de threads
  - Recomendados para tarefas não bloqueantes ou intensivas de processamento (CPU-bound)

Threads de Plataforma

- Arquitetura
  - Executados por Threads do Sistema Operacional (kernel threads)
  - Escalonados diretamente pelo escalonador do sistema operacional
  - Adotam mapeamento 1:1
    - Cada thread de plataforma é vinculado a um único thread do sistema operacional
  - Consomem bastante recursos de memória
    - Cada thread de plataforma adota um stack frame de tamanho fixo para manter dados requeridos à execução (default: 1Mb)

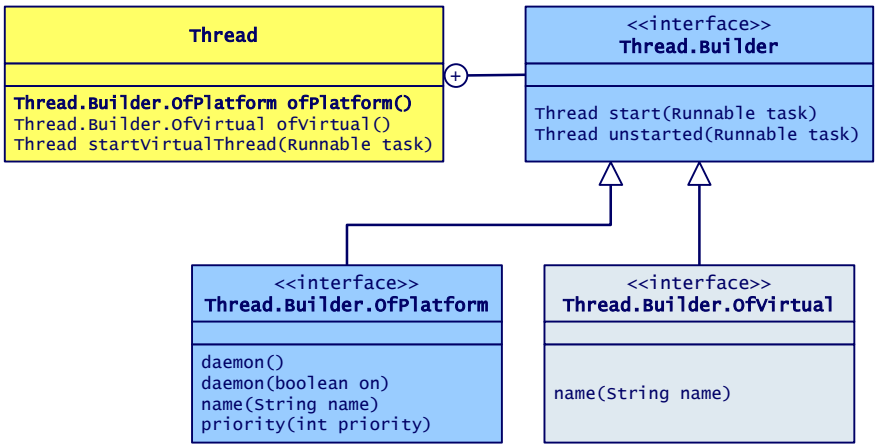
Threads de Plataforma

- Arquitetura



Threads de Plataforma

- Hierarquia de Classes



Threads de Plataforma

- Criando Threads
  - Primeiro, ativar o método `Thread.ofPlatform()`
    - Retorna um construtor que implementa a interface `Thread.Builder.OfPlatform`
  - Depois, pode-se criar o thread de duas formas:
    - Ativar o método `start()` do construtor de threads para instanciar e iniciar o novo thread
    - Ativar o método `unstarted()` do construtor de threads para apenas instanciar o novo thread

```
Runnable r = ...
Thread t = Thread.ofPlatform().start(r);
```

```
Runnable r = ...
Thread t = Thread.ofPlatform().unstarted(r);
t.start();
```

Threads de Plataforma

- Criando Threads

```
public class HelloWorldRun implements Runnable {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldRun h = new HelloWorldRun();
    Thread t = new Thread(h);
    t.start();
  }
}
```

```
public class HelloWorldRun implements Runnable {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldRun h = new HelloWorldRun();
    Thread t = Thread.ofPlatform().start(h);
  }
}
```


Threads de Plataforma

- Criando Threads

```
class MyRun implements Runnable {
  private String message;
  public MyRun(String m) {message = m;}
  public void run() {
    for(int r = 0; r < 10; r++)
      System.out.println(message);
  }
}

public class PlatformThreadDemo {
  public static void main(String[] args) {
    MyRun r1 = new MyRun("Primeiro Thread");
    MyRun r2 = new MyRun("Segundo Thread");
    Thread t1 = Thread.ofPlatform().start(r1);
    Thread t2 = Thread.ofPlatform().start(r2);
  }
}
```

Threads de Plataforma

- Criando Threads

```
Runnable r = () -> { System.out.println("Hello World"); }
Thread t = Thread.ofPlatform().start(r);
```

```
Thread t = Thread.ofPlatform().start(() -> {
   System.out.println("Hello World");
});
```

```
class MyClass {
  public void hello() { System.out.println("Hello World") };
}
...
MyClass obj = new MyClass();
Thread t = Thread.ofPlatform().start(obj::hello);
```

```
class MyClass {
  public static void hello() { System.out.println("Hello World") };
}
...
Thread t = Thread.ofPlatform().start(MyClass::hello);
```


Threads de Plataforma

- Configurando Threads
  - Nome é atribuído automaticamente na criação
    - Pode ser configurado com o método `Thread.Builder.OfPlatform.name()`
  - Prioridade é herdada do thread pai
    - Pode ser configurada com o método `Thread.Builder.OfPlatform.priority()`

```
Runnable r = ...
Thread t = Thread.ofPlatform().name("MyThread")
                              .priority(5).
                              .start(r);
```

Threads de Plataforma

- Configurando Threads
  - Modo de execução é herdado do thread pai
    - Pode executar como daemon ou non-daemon
    - Thread main executa em modo non-daemon
    - Pode ser configurado com o método Thread.Builder.OfPlatform.daemon()
      - daemon(): configura modo daemon
      - daemon(boolean on): configura modo daemon (true) / non-daemon (false)

```
Runnable r = ...
Thread t = Thread.ofPlatform().daemon().start(r);
```

```
Runnable r = ...
Thread t = Thread.ofPlatform().daemon(true).start(r);
```


Threads Virtuais

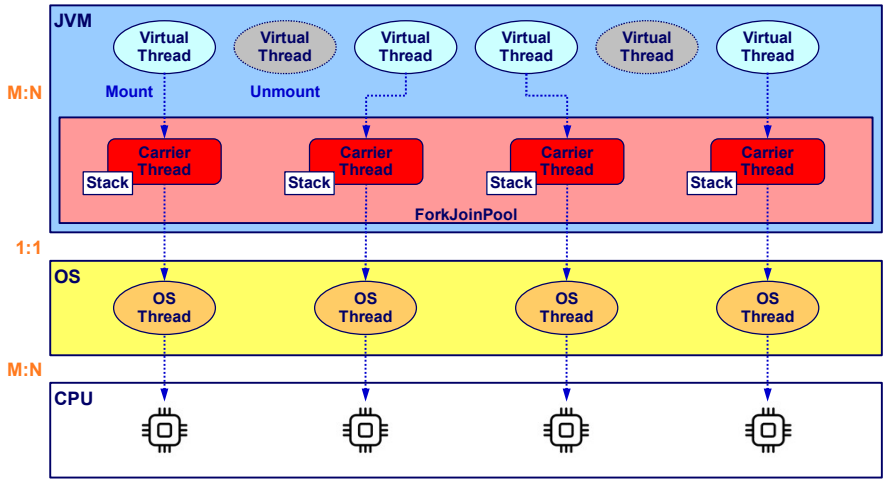
- Características
  - Representam os novos tipos de threads da arquitetura Java
  - Não muito adequados à execução de todo tipo de tarefa
  - Requer bem menos recursos do que threads de plataforma
    - Menor consumo de memória e processamento
    - Suporte a alguns milhões de threads
  - Recomendados para tarefas bloqueantes ou intensivas de entrada/saída (I/O bound)

Threads Virtuais

- Arquitetura
  - Executados por um pool interno de threads de plataforma (platform threads)
  - Escalonados diretamente pela JVM
  - Adotam mapeamento M:N
    - Muitos threads virtuais são dinamicamente vinculados a poucos threads de plataforma
  - Consomem poucos recursos de memória
    - Threads virtuais não adotam stack frame para manter dados requeridos à execução
    - Mantêm dados de tamanho dinâmico na memória heap da JVM

Threads Virtuais

- Arquitetura



Threads Virtuais

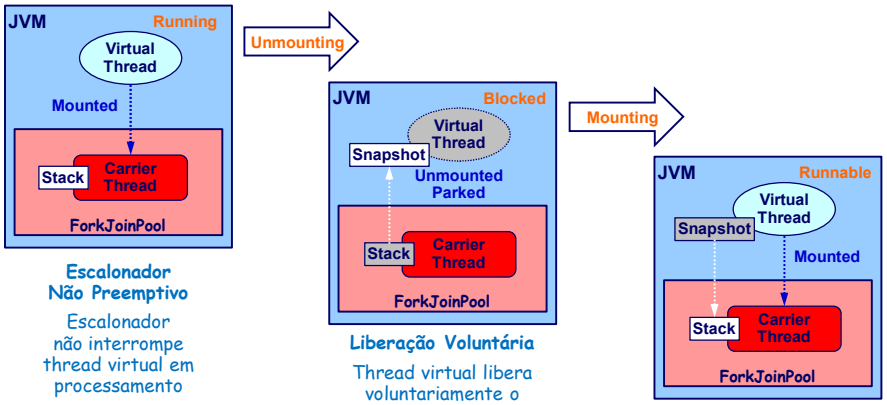
- Arquitetura
  - Threads virtuais são dinamicamente vinculados a threads de transporte (carrier threads)
  - Threads de transporte compõem um pool de threads de plataforma do tipo ForkJoinPool
    - Criado de forma automática pela JVM
    - Especializado para executar tarefas individuais com a estratégia dividir para conquistar
    - Utiliza algoritmo eficiente de roubo de trabalho (work-stealing)

Threads Virtuais

- Arquitetura
  - Adota algoritmo de escalonamento cooperativo
    - Operações bloqueantes e de entrada/saída dispararam a montagem/desmontagem (mounting/unmounting) entre threads virtuais e threads de transporte
      - Thread virtual não tem conhecimento do thread de transporte subjacente em que está montado
    - Códigos nativos (JNI) bloqueantes e de entrada/saída disparam a fixação/desfixação (pinning/unpinning) entre threads virtuais e threads de transporte

Threads Virtuais

- Escalonamento Cooperativo



Escalonador Não Preemptivo

Escalonador não interrompe thread virtual em processamento

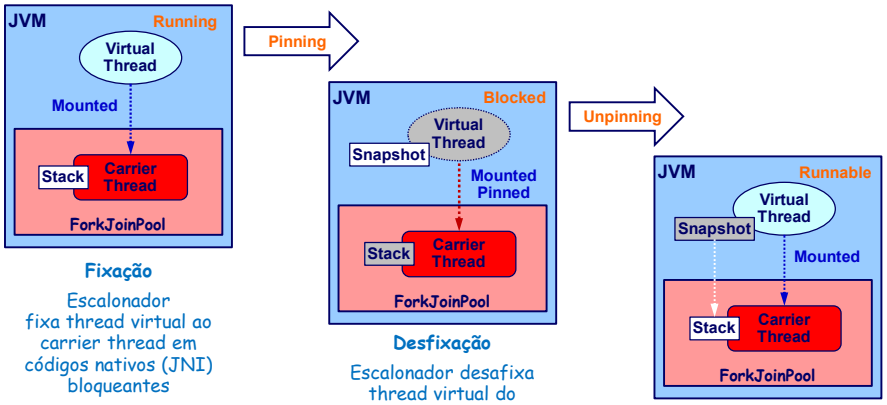
Não divide o tempo em intervalos (slots)

Liberação Voluntária

Thread virtual libera voluntariamente o carrier thread ao encontrar operação bloqueante (I/O, sleep)

Threads Virtuais

- Escalonamento Cooperativo



Fixação

Escalonador fixa thread virtual ao carrier thread em códigos nativos (JNI) bloqueantes

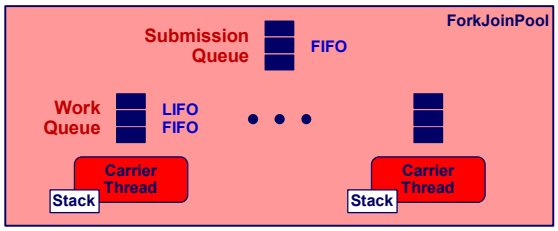
Java 24+ não fixa com synchronized

Desfixação

Escalonador desafixa thread virtual do carrier thread ao identificar término da operação bloqueante

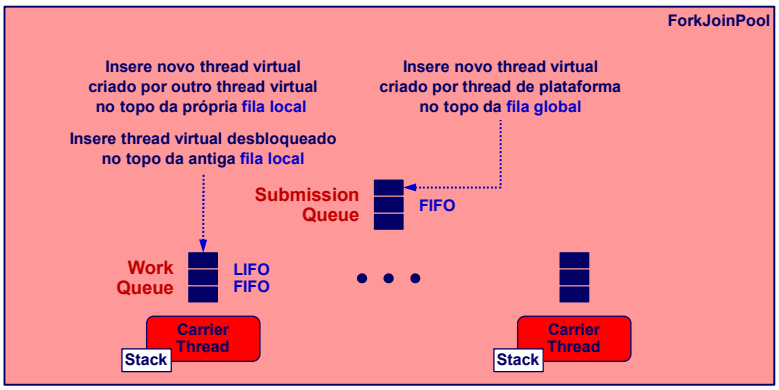
Threads Virtuais

- Arquitetura de Filas
  - Submission Queue
    - Fila global que mantém novos threads virtuais
  - Work Queue
    - Fila local que mantém threads virtuais prontos para executar
    - Cada carrier thread tem sua própria fila local



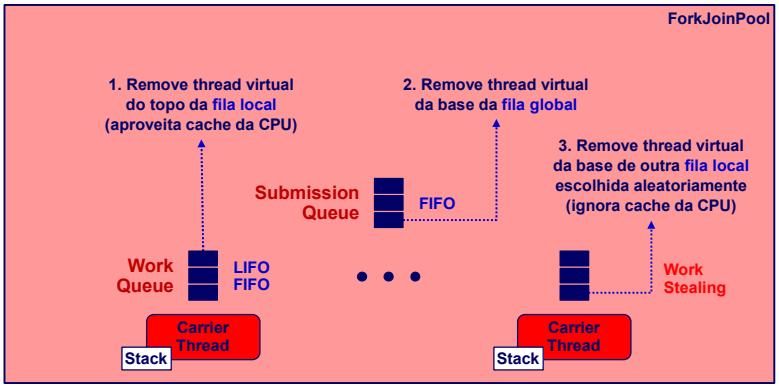
Threads Virtuais

- Arquitetura de Filas
  - Inserção



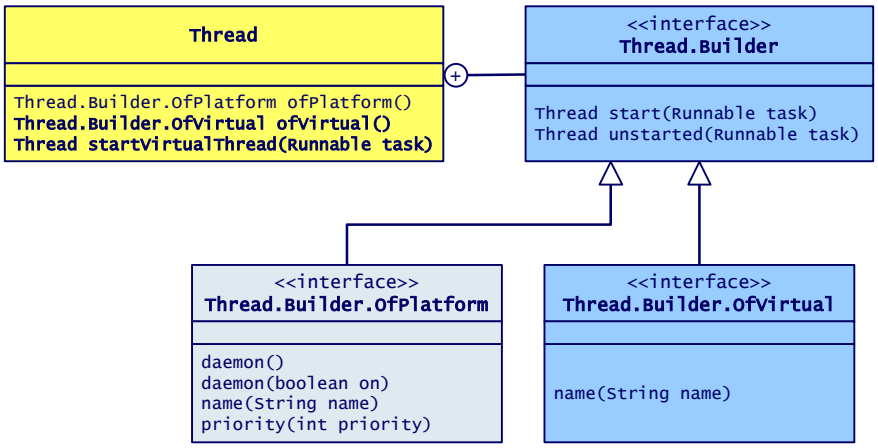
Threads Virtuais

- Arquitetura de Filas
  - Remoção



Threads Virtuais

- Hierarquia de Classes



Threads Virtuais

- Criando Threads
  - Primeiro, ativar o método `Thread.ofVirtual()`
    - Retorna um construtor que implementa a interface `Thread.Builder.OfVirtual`
  - Depois, pode-se criar o thread de duas formas:
    - Ativar o método `start()` do construtor de threads para instanciar e iniciar o novo thread
    - Ativar o método `unstarted()` do construtor de threads para apenas instanciar o novo thread

```
Runnable r = ...
Thread t = Thread.ofVirtual().start(r);
```

```
Runnable r = ...
Thread t = Thread.ofVirtual().unstarted(r);
t.start();
```


Threads Virtuais

- Criando Threads
  - Pode-se criar e iniciar um thread sem usar um construtor
  - Basta ativar diretamente o método `Thread.startVirtualThread()`

```
Runnable r = ...
Thread t = Thread.startVirtualThread(r);
```

Threads Virtuais

- Criando Threads

```
public class HelloWorldRun implements Runnable {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldRun h = new HelloWorldRun();
    Thread t = new Thread(h);
    t.start();
  }
}
```

```
public class HelloWorldRun implements Runnable {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldRun h = new HelloWorldRun();
    Thread t = Thread.ofVirtual().start(h);
    t.join();
  }
}
```


Threads Virtuais

- Criando Threads

```
class MyRun implements Runnable {
  private String message;
  public MyRun(String m) {message = m;}
  public void run() {
    for(int r = 0; r < 10; r++)
      System.out.println(message);
  }
}

public class VirtualThreadDemo {
  public static void main(String[] args) {
    MyRun r1 = new MyRun("Primeiro Thread");
    MyRun r2 = new MyRun("Segundo Thread");
    Thread t1 = Thread.ofVirtual().start(r1);
    Thread t2 = Thread.startVirtualThread(r2);
    t1.join();
    t2.join();
  }
}
```

Threads Virtuais

- Criando Threads

```
Runnable r = () -> { System.out.println("Hello World"); }
Thread t = Thread.ofVirtual().start(r);
```

```
Thread t = Thread.ofVirtual().start(() -> {
   System.out.println("Hello World");
});
```

```
class MyClass {
  public void hello() { System.out.println("Hello World") };
}
...
MyClass obj = new MyClass();
Thread t = Thread.ofVirtual().start(obj::hello);
```

```
class MyClass {
  public static void hello() { System.out.println("Hello World") };
}
...
Thread t = Thread.ofVirtual().start(MyClass::hello);
```


Threads Virtuais

- Criando Threads
  - Pode-se adotar um pool de threads que cria e inicia uma thread virtual para cada tarefa enviada
    - Executados pelo pool interno de threads de plataforma do tipo ForkJoinPool
  - Basta ativar diretamente o método estático `Executors.newVirtualThreadPerTaskExecutor()`

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
  executor.submit(() -> { return "Tarefa"; });
}
```

Threads Virtuais

- Configurando Threads
  - Nome não é atribuído por default na criação
    - Pode ser configurado com o método `Thread.Builder.OfVirtual.name()`
  - Prioridade é sempre fixa - `NORM_PRIORITY (5)`
    - Não pode ser alterada
  - Modo de execução é sempre daemon
    - Não pode ser alterado
    - Não impede o término da aplicação

```
Runnable r = ...
Thread t = Thread.ofVirtual().name("MyThread")
                             .start(r);
```


Threads Virtuais

- Configurando o Pool
  - `jdk.virtualThreadScheduler.parallelism`
    - Número de threads de plataforma ativos que podem executar simultaneamente
    - Default: número de núcleos disponíveis
  - `jdk.virtualThreadScheduler.maxPoolsize`
    - Define o limite máximo absoluto de threads de plataforma no `ForkJoinPool`
    - Default: 256

```
> java -Djdk.virtualThreadScheduler.parallelism=4
       -Djdk.virtualThreadScheduler.maxPoolSize=128 MyApp
```

```
System.setProperty("jdk.virtualThreadScheduler.parallelism", "4");
System.setProperty("jdk.virtualThreadScheduler.maxPoolSize", "128");
```

Threads Virtuais

- Estados do Thread
  - Estados Públicos
    - Visíveis externamente pelo método `Thread.getState()`
    - Definidos pelo tipo enumerado `Thread.State`
    - Permitem manter compatibilidade com ferramentas de monitoramento
  - Estados Internos
    - Não são visíveis externamente
    - Representam situações relacionadas com:
      - Vincular e desvincular threads de transporte
      - Salvar e restaurar dados entre heap e stack

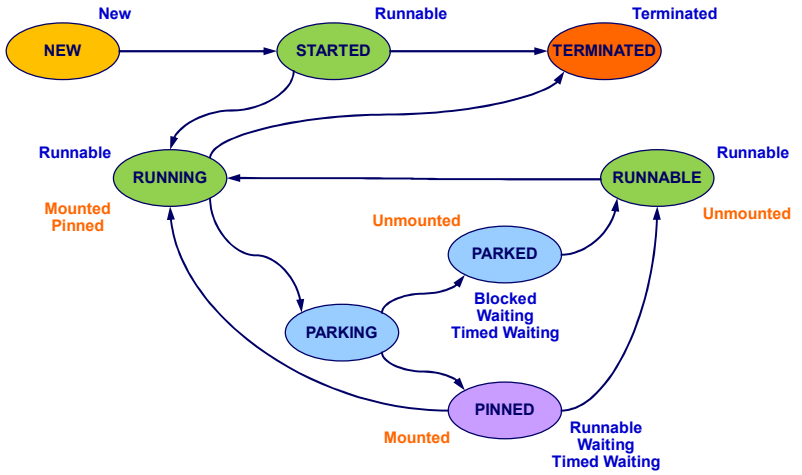
Threads Virtuais

- Estados Internos x Públicos

Estado Interno (JVM)	Estado Público (Thread.State)	Descrição
NEW	NEW	Instanciado na memória heap, mas método `start()` ainda não foi chamado
MOUNTED	RUNNABLE	Montado ativamente no thread de transporte Pode estar executando ou pronto para executar
UNMOUNTED	RUNNABLE	Está pronto para executar Desmontado do thread de transporte Aguarda na fila do escalonador JVM
PARKED	WAITING TIME WAITING	Realizou operação bloqueante Estado foi salvo na memória heap Thread de transporte foi liberado
PINNED	RUNNABLE BLOCKED WAITING	Realizou operação bloqueante em código nativo Thread de transporte não pode ser desmontado
TERMINATED	TERMINATED	Execução terminou e aguarda coletor de lixo

Threads Virtuais

- Estados Internos



https://rockthejvm.com/articles/the-ultimate-guide-to-java-virtual-threads

Threads Virtuais

- Estados Públicos

